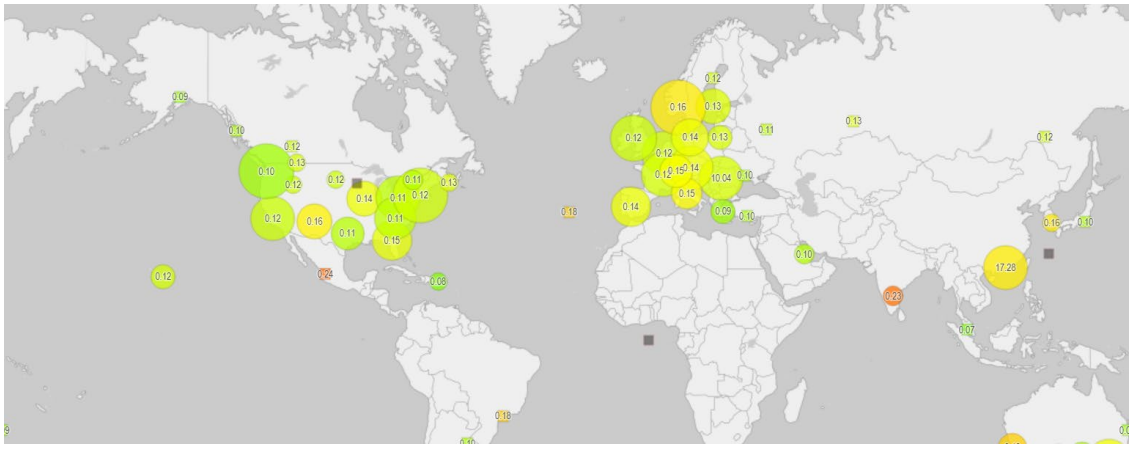
IIOT for Advanced Manufacturing (ECMC104)

Lab 8 – Configuring a Data Visualisation Package

Objectives

1. To install and configure the open-source InfluxDB, which is a time-series database.
2. To install and configure the open-source Grafana, which is a web-based data visualisation package.
3. To configure Grafana with alert monitoring on data.

IoT Case Study: Global Environmental Monitoring



"uRADMonitor is a project that addresses pollution, the kind that we are unable to see, but directly affects our health and can cause life-threatening diseases. To help fight pollution, we have designed several fixed and mobile detectors, packed with powerful sensors, capable of detecting both the chemical and the harmful physical factors. All data goes online to a global network, so anyone can access it freely, at any time."

Source:
https://cleantechnica.com/2015/12/11/uradmonitor-global-pollution-tracking-network/
https://www.uradmonitor.com/

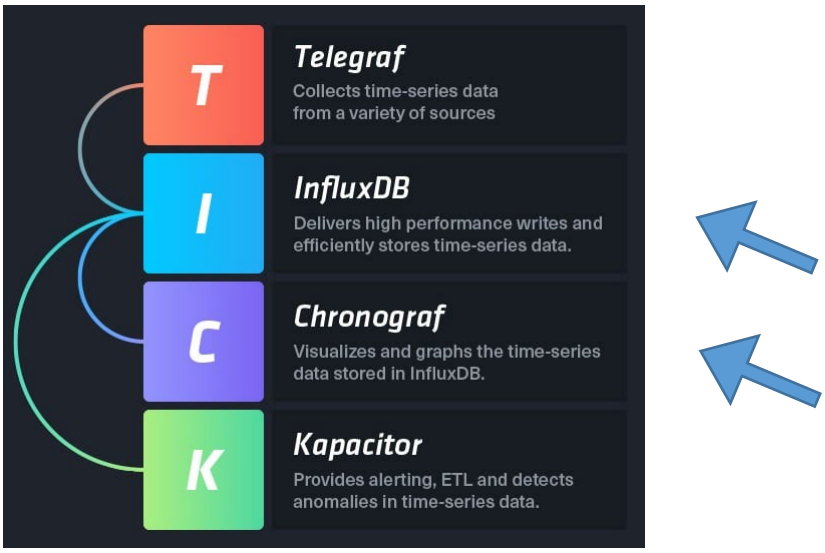
Carry out a research to find out:

- What is the purpose of Data Visualization in Internet of Things(IoT)?
- Explain the importance of the role that Data Visualization plays in the uRADMonitor project. How would it be like if there is no element of Data Visualization in this project?

Contents

1 Introduction to TICK

InfluxDB (I) is part of the "**TICK**" stack. The complete stack offers a complete solution to IoT applications. In this exercise, besides InfluxDB, we will also be exploring the **C** (for **Chronograf**) of TICK. This is a visualisation application. For this exercise, we will be using **Grafana** instead of Chronograf as Grafana has added advantages to use different databases, besides InfluxDB.



InfluxDB is part of the "**TICK**" stack

2 InfluxDB for Time-Series Data Storage

References:
https://docs.influxdata.com/influxdb/v2.4/

2.1 Download and Install InfluxDB Package

Launch the WSL2/Ubuntu Terminal.

Create a folder, **/home/<user>/Develop/Lab7** folder. This folder will store all your download and program codes for this lab.

Download the InfluxDB package from the web with the following command:

```
$ wget https://dl.influxdata.com/influxdb/releases/influxdb2-2.4.0-amd64.deb --no-check-certificate
```

* You may check for the latest version of InfluxDB at the download page and amend the command accordingly: https://portal.influxdata.com/downloads.

You should observe an output similar to the following if the package is successfully downloaded:

```
--2022-12-12 14:57:10--  https://dl.influxdata.com/influxdb/releases/influxdb2-2.4.0-amd64.deb
```

```
Resolving dl.influxdata.com (dl.influxdata.com)... 143.204.9.73, 143.204.9.2,
143.204.9.88, ...
Connecting to dl.influxdata.com (dl.influxdata.com)|143.204.9.73|:443...
connected.
WARNING: cannot verify dl.influxdata.com's certificate, issued by ‘CN=Zscaler
Intermediate Root CA (zscaler.net) (t)\\ ,OU=Zscaler Inc.,O=Zscaler
Inc.,ST=California,C=US’:
  Unable to locally verify the issuer's authority.
HTTP request sent, awaiting response... 200 OK
Length: 92102818 (88M) [application/vnd.debian.binary-package]
Saving to: ‘influxdb2-2.4.0-amd64.deb’

influxdb2-2.4.0-amd64.deb
100%[=============================================================>]  87.84M
3.02MB/s    in 17s

2022-12-12 14:57:34 (5.17 MB/s) - ‘influxdb2-2.4.0-amd64.deb’ saved
[92102818/92102818]
```

Next, install the package:

```
$ sudo dpkg -I influxdb2-2.4.0-amd64.deb
```

You should observe the following output:

```
Selecting previously unselected package influxdb2.
(Reading database ... 42152 files and directories currently installed.)
Preparing to unpack influxdb2-2.4.0-amd64.deb ...
Unpacking influxdb2 (2.4.0-1) ...
Setting up influxdb2 (2.4.0-1) ...
Created symlink /etc/systemd/system/influxd.service →
/lib/systemd/system/influxdb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/influxdb.service →
/lib/systemd/system/influxdb.service.

After installation, the output observes will be:
new Debian package, version 2.0.
 size 92102818 bytes: control archive=2499 bytes.
      26 bytes,     1 lines      conffiles
     317 bytes,    14 lines      control
     481 bytes,     7 lines      md5sums
    3852 bytes,   142 lines   *  postinst             #!/bin/bash
    1475 bytes,    58 lines   *  postrm               #!/bin/bash
     428 bytes,    22 lines   *  preinst              #!/bin/bash
 Package: influxdb2
 Version: 2.4.0-1
 License: MIT
 Vendor: InfluxData
 Architecture: amd64
 Maintainer: support@influxdb.com
 Installed-Size: 146654
 Depends: curl
 Conflicts: influxdb
 Recommends: influxdb2-cli
 Section: default
 Priority: extra
 Homepage: https://influxdata.com
 Description: Distributed time-series database.
```

2.2 Configuration

Add the InfluxData repository (you can try to understand this as being part of the required configuration) with the following commands:

```
$ sudo curl -sL https://repos.influxdata.com/influxdb.key | sudo apt-key add -

$ sudo echo "deb https://repos.influxdata.com/ubuntu bionic stable" | sudo tee
/etc/apt/sources.list.d/influxdb.list
```

Hint:
The first command requires the ***curl*** utility. You will be asked to install with the following command if ***curl*** has not been installed yet:

```
$ sudo apt install curl
```

Upon completing all the 3 commands above, this will create a file called "/etc/apt/sources.list.d/influxdb.list" if it worked.

Next, install and start the InfluxDB service:

```
$ sudo apt-get update
$ sudo apt-get install influxdb
$ sudo influxd &
```

To check whether the service is started, type the following command:

```
$ sudo service influxdb status
```

This will give you the following output if InfluxDB is started successfully;

```
● influxdb.service - InfluxDB is an open-source, distributed, time series database
     Loaded: loaded (/lib/systemd/system/influxdb.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-01-05 08:49:38 +08; 1h 25min ago
       Docs: man:influxd(1)
   Main PID: 199 (influxd)
      Tasks: 17 (limit: 9138)
     Memory: 64.4M
        CPU: 13.057s
     CGroup: /system.slice/influxdb.service
             └─199 /usr/bin/influxd -config /etc/influxdb/influxdb.conf
```

Install InfluxDB client:

```
$ sudo apt install influxdb-client
```

2.3 Testing InfluxDB

To test that the installation is done correctly, connect to **InfluxDB** with this command:

```
$ influx
```

The following output should be observed:

```
Connected to http://localhost:8086 version 1.6.7~rc0
InfluxDB shell version: 1.6.7~rc0
>
```

Type **quit** to exit from the InfluxDB command line prompt:

```
> quit
```

2.4 Install InfluxDB Python Client

References:

https://github.com/influxdata/influxdb-python
https://influxdb-python.readthedocs.io/en/latest/

InfluxDB works with a number of languages – one of them is Python. In order to use Python, it is necessary to install the **InfluxDB-Python** client library or package.

Run the following command to install this package:

```
$ sudo apt install python3-influxdb
```

2.5 Write a InfluxDB Python Program

With all the above set up, you are now ready to write a simple program to perform writing to the InfluxDB database.

2.5.1 Create a Database

Launch the **Terminal** program. Run the InfluxDB command line program with the following command:

```
$ influx
```

```
Connected to http://localhost:8086 version 1.5.1
InfluxDB shell version: 1.5.1
>
```

At the prompt, type:

```
> help
```

This shows you a condensed list of commands you can execute:

```
> help
Usage:
        connect <host:port>   connects to another node specified by host:port
        auth                  prompts for username and password
        pretty                toggles pretty print for the json format
        chunked               turns on chunked responses from server
        chunk size <size>     sets the size of the chunked responses.
                                        Set to 0 to reset to the default chunked size
        use <db_name>         sets current database
        format <format>       specifies the format of the server responses: json, csv, or column
        precision <format>    specifies the format of the timestamp: rfc3339, h, m, s, ms, u or ns
        consistency <level>   sets write consistency level: any, one, quorum, or all
        history               displays command history
        settings              outputs the current settings for the shell
        clear                 clears settings such as database or retention policy.
                                        run 'clear' for help
        exit/quit/ctrl+d      quits the influx shell
```

```
        show databases        show database names
        show series           show series information
        show measurements     show measurement information
        show tag keys         show tag key information
        show field keys       show field key information

        A full list of influxql commands can be found at:
        https://docs.influxdata.com/influxdb/latest/query_language/spec/
```

At the prompt, type:

```
> show databases
```

This gives you the current databases which have been created so far.

```
name: databases
name
----
_internal
>
```

To create a new database with the name **mydb**, type:

```
> create database mydb
```

Type the following again to confirm that it has been created:

```
> show databases
```

You should see the following output:

```
name: databases
name
----
_internal
mydb
>
```

Type **quit** to exit from the InfluxDB command line prompt:

```
> quit
```

2.5.2 Write the Program

Create the folder, **/home/<user>/Develop/Lab7**. This will be the folder where you will store all your program codes for this lab.

Exercise 1:

Edit the following program and save it as **writedb.py**.

```
import argparse
from influxdb import InfluxDBClient
from influxdb.client import InfluxDBClientError
import datetime
import time
import random

USER = 'root'
PASSWORD = 'root'
DBNAME = 'mydb'
HOST = 'localhost'
PORT = 8086
dbclient = None;

def main():
    dbclient = InfluxDBClient(HOST, PORT, USER, PASSWORD, DBNAME)
    while True:
        data_point = getSensorData()
        dbclient.write_points(data_point)
        print("Written data")
        time.sleep(2)


def getSensorData():
    sensorData = random.randint(15, 40)
    now = time.gmtime()
    pointValues = [
       {
          "time": time.strftime("%Y-%m-%d %H:%M:%S", now),
          "measurement": 'reading',
          "tags": {
              "nodeId": "node_1",
          },
          "fields": {
              "value": sensorData
          },
       }
    ]

    return(pointValues)

if __name__ == '__main__':
    main()
```

writedb.py

The following points explains the important parts of the program:

- **Creating a Client Object:**
  The function, **InfluxDBClient()** is used to create a client instance.

Explore this library function:

```
class influxdb.InfluxDBClient(host=u'localhost', port=8086, username=u'root',
password=u'root', database=None, ssl=False, verify_ssl=False, timeout=None,
retries=3, use_udp=False, udp_port=4444, proxies=None)
```

For more details on this function - **InfluxDBClient()**, go to the URL below:
https://influxdb-python.readthedocs.io/en/latest/api-documentation.html#influxdbclient

Question:

1) Using what you have learnt in **Lab 5** on named and optional arguments in a Python function, how many optional arguments are there in this function?

2) What is the default value for the argument, **host**?

3) What is the default value for the argument, **port**?

- **Writing Data:**
  The function, **write_points()** is used to write data into the database.

Explore this library function:

```
write_points(points, time_precision=None, database=None, retention_policy=None,
tags=None, batch_size=None, protocol=u'json')
```

For more details on this function - **write_points(),** go to the URL below:
https://influxdb-python.readthedocs.io/en/latest/api-documentation.html#influxdb.InfluxDBClient.write_points

It is important to note that the first parameter, **`points`** is a Python **list**. More specifically, it is a **list** of dictionaries. Each dictionary represents a point to be written. In other words, **`points`** refers to a list of points to be written to the database.

Question:

1) Using what you have learnt in **Lab 5** on named and optional arguments in a Python function, how many optional arguments are there in this function?

2) What is the default value for the argument, **protocol**?

- **JSON Data Structure:**

As you have seen in **Lab 4 Section 2.2**, **pointValues** is a **JSON data structure**.

```
pointValues = [
       {
          "time": time.strftime("%Y-%m-%d %H:%M:%S", now),
          "measurement": 'reading',
          "tags": {
              "nodeId": "node_1",
          },
          "fields": {
              "value": sensorData
          },
       }
    ]
```

Fill in the blanks below. Refer to **Lab 2 – Python & Raspberry Pi**.

Questions:

1) In the above, **pointValues** is a Python _____ object.

2) However, **pointValues** contain only _____ element in the _____ object.

3) This _____ is a _____ object.

4) This _____ object contains _____ main key-value pairs, with the keys as "time", "measurement", "tags" and "fields". This _____ object is a _____ with the _____ "tags" and "fields".

- **InfluxDB Key Concepts:**

Please read the following to understand the key concepts of InfluxDB.

Reference:
https://docs.influxdata.com/influxdb/v2.4/reference/key-concepts/

Questions:

1) As InfluxDB is a time-series database, it is necessary to have the _____ column. All InfluxDB data has this column.

2) The **measurement** is conceptually similar to a database _____.

3) _____ values are the data. They can be string, float, integers or Boolean.

4) The _____ in the above example is _____. And the **field-value** is the variable - **sensorData**.

5) As tags are indexed, this makes queries faster. It is optional to use tags. In the above example, the **tag-key** is the _____; while the **tag-value** is **node_1**.

Note: This example illustrates the sensor data from **node_1**. In a scaled-up example, sensor data can come from multiple nodes. It is important to know which node does it comes from.

2.5.3 Testing the Program

Run the program with the following command:

```
$ python3 writedb.py
```

https://docs.influxdata.com/influxdb/v2.4/
You may refer to the link for for the latest update.

We can either enter the InfluxDB interactive prompt to view the data, or execute a command at the command line using the command line interface (CLI) described above to view the data. If using the CLI, you can execute:

```
$ influx -database 'mydb' -execute 'select * from reading'
```

You can also export the readings to a file (in this case, **mydata.csv** in csv format) by executing the following command:

```
$ influx -database 'mydb' –execute 'select * from reading' –format 'csv' >>
mydata.csv
```

3 Grafana for Dashboard and Metrics Monitoring

References:

http://docs.grafana.org/installation/debian

Grafana is a visualisation package that can be configured to work with InfluxDB.

3.1 Install Package

First, open the **Terminal** program and path to '**/home/<user>/Develop/Lab7**' you created earlier.

Download the Grafana package from the web with the **wget** command:

```
$ wget https://dl.grafana.com/oss/release/grafana_9.3.1_amd64.deb --no-check-
certificate
```

At the point of writing, the version of Grafana is 9.3.1.

The following will be observed. The downloading may take a few minutes depending on your internet speed.

```
Resolving dl.grafana.com (dl.grafana.com)... 151.101.54.217, 2a04:4e42:d::729
Connecting to dl.grafana.com (dl.grafana.com)|151.101.54.217|:443... connected.
ERROR: cannot verify dl.grafana.com's certificate, issued by ‘CN=Zscaler Intermediate Root CA
(zscaler.net) (t)\\ ,OU=Zscaler Inc.,O=Zscaler Inc.,ST=California,C=US’:
  Unable to locally verify the issuer's authority.
To connect to dl.grafana.com insecurely, use `--no-check-certificate'.
iotp@ENG-R90YT61T-10:~/Develop/lab6/grafana$ wget
https://dl.grafana.com/oss/release/grafana_9.3.1_amd64.deb --no-check-certificate
--2022-12-12 16:38:39--  https://dl.grafana.com/oss/release/grafana_9.3.1_amd64.deb
Resolving dl.grafana.com (dl.grafana.com)... 151.101.54.217, 2a04:4e42:d::729
Connecting to dl.grafana.com (dl.grafana.com)|151.101.54.217|:443... connected.
WARNING: cannot verify dl.grafana.com's certificate, issued by ‘CN=Zscaler Intermediate Root
CA (zscaler.net) (t)\\ ,OU=Zscaler Inc.,O=Zscaler Inc.,ST=California,C=US’:
  Unable to locally verify the issuer's authority.
HTTP request sent, awaiting response... 200 OK
Length: 88725002 (85M) [application/octet-stream]
Saving to: ‘grafana_9.3.1_amd64.deb’

grafana_9.3.1_amd64.deb
100%[=====================================================================>]  84.61M  5.36MB/s
in 48s

2022-12-12 16:39:33 (1.75 MB/s) - ‘grafana_9.3.1_amd64.deb’ saved [88725002/88725002]
```

Next, install Grafana with the following 2 commands:

```
$ sudo apt-get install -y adduser libfontconfig
$ sudo dpkg -i grafana_9.3.1_amd64.deb
```

3.2 Starting the Grafana Service

Start the Grafana server:

```
$ sudo service grafana-server start
```

Configure the service daemon in the operating system so that the Grafana service starts every time upon booting up:

```
$ sudo systemctl enable grafana-server.service
```

This is what you will observe:

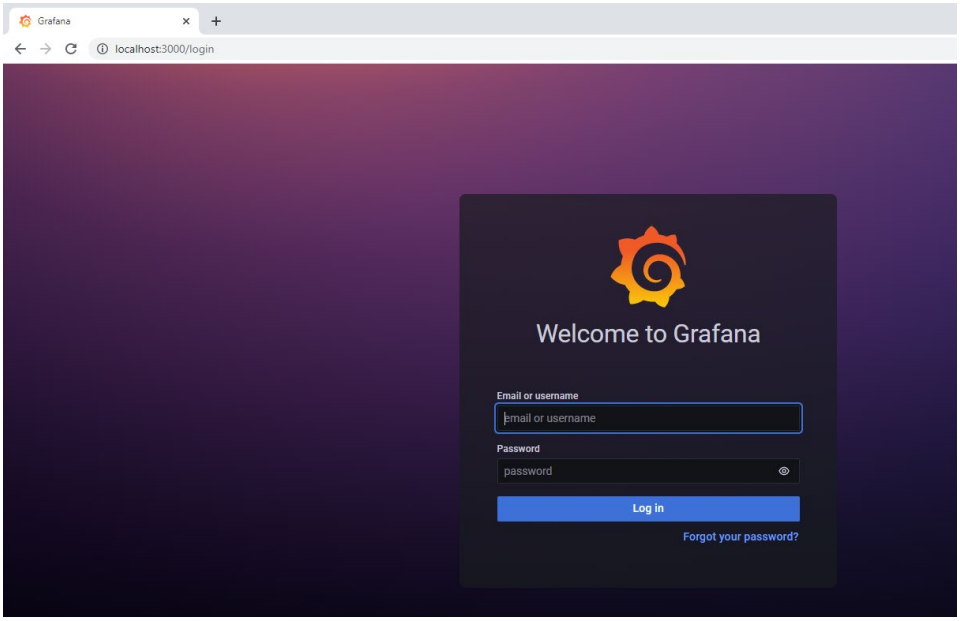
```
$ sudo systemctl enable grafana-server.service
Synchronizing state of grafana-server.service with SysV service script with
/lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable grafana-server
$
```

3.3 Setting up Grafana to read from InfluxDB

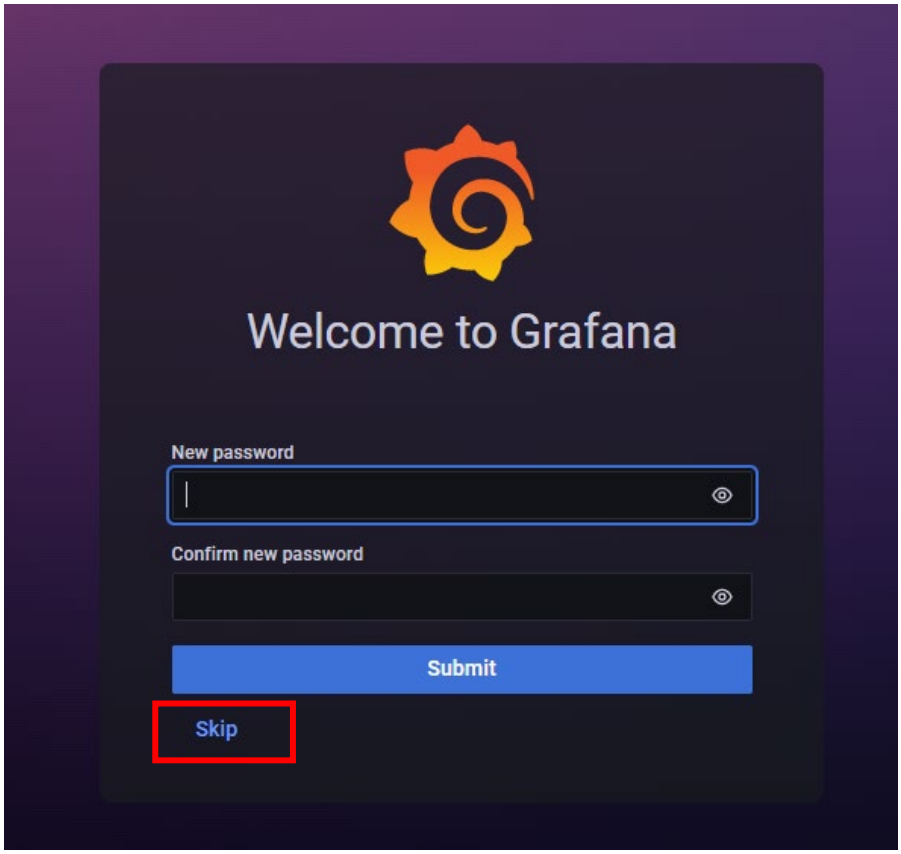
References:
https://grafana.com/docs/grafana/latest/

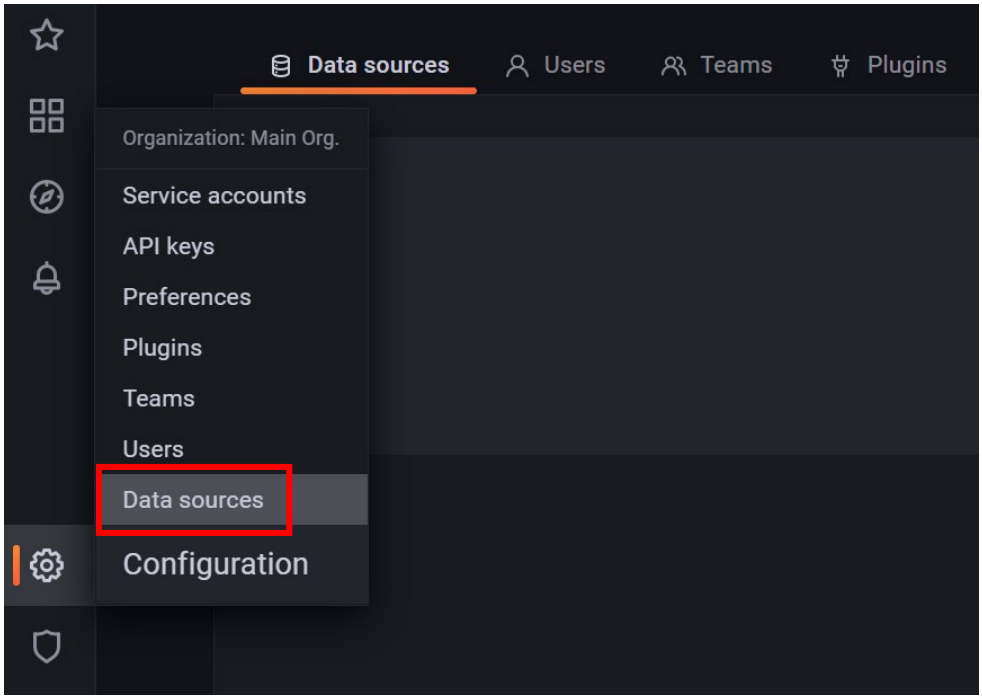
In the **Chrome** browser, go to **localhost:3000**. Grafana uses port 3000 to serve its web pages.

Log in as (user) **admin**, and (password) **admin**.

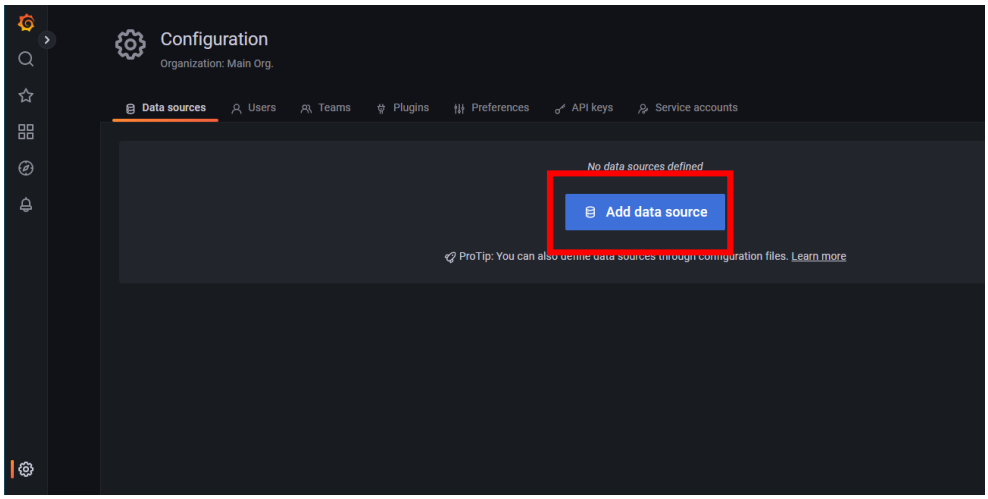


When ask to change new password, **Skip** for now.

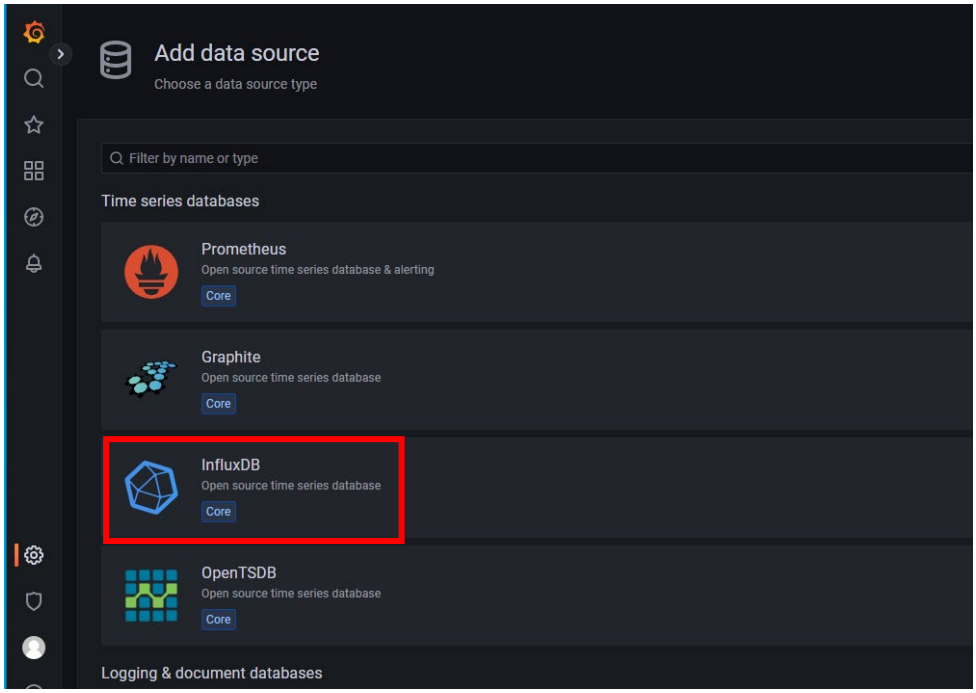




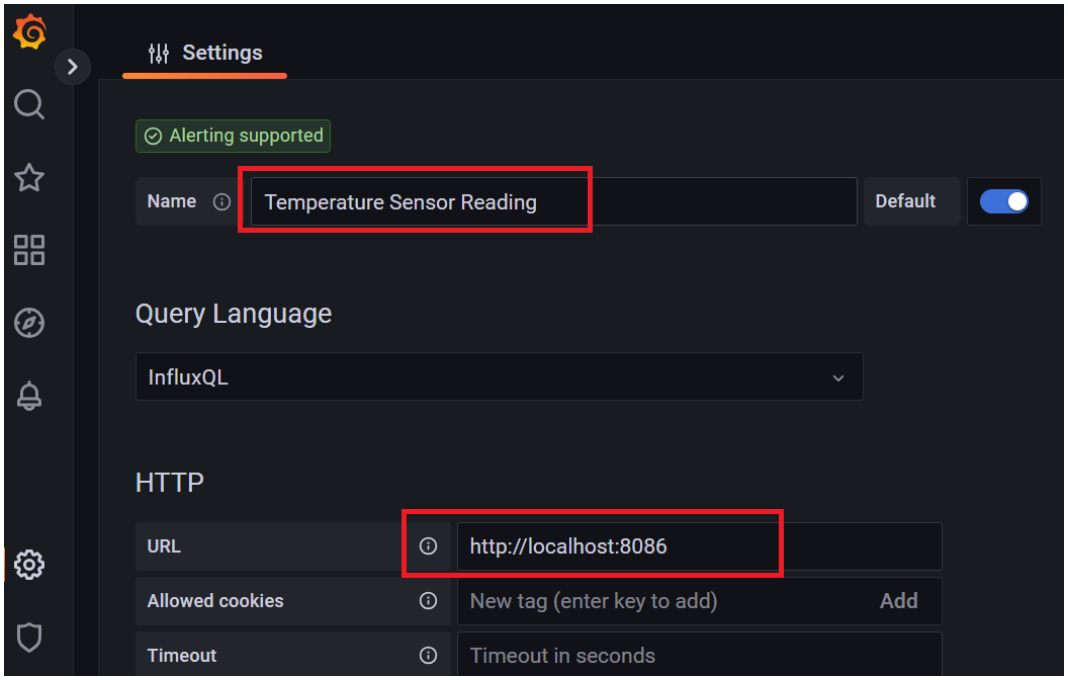
Click the  icon and **Data sources**. This brings you to the following settings page:



Click **Add data source**:



Select InfluxDB:

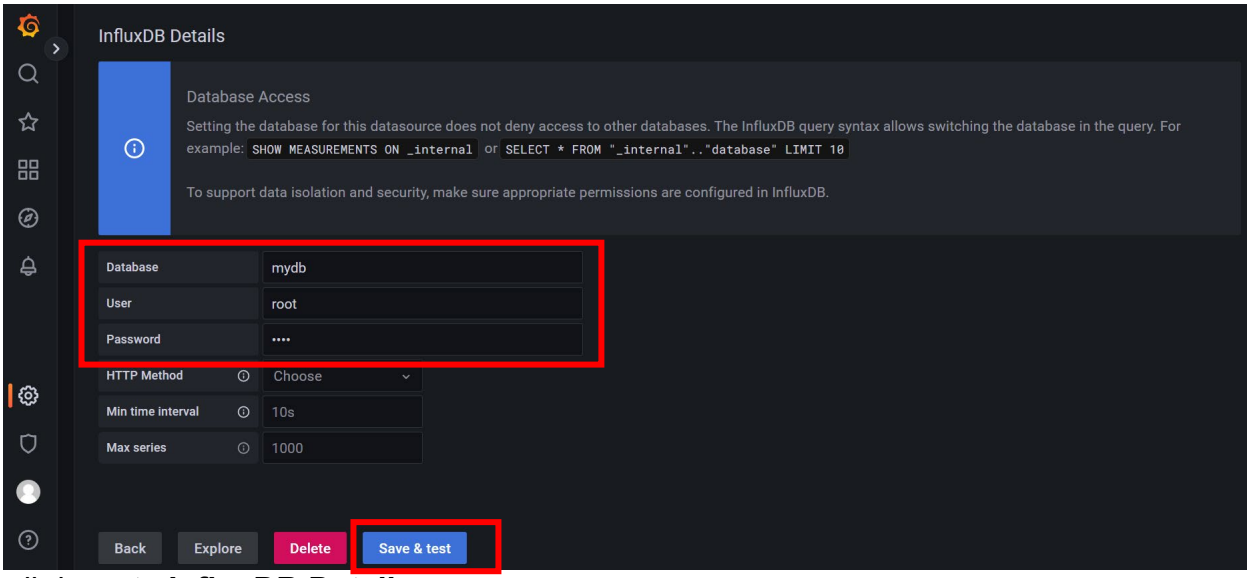


Under the **Settings** tab, enter the following information for the appropriate fields:

- Name: Temperature Sensor Reading

HTTP:

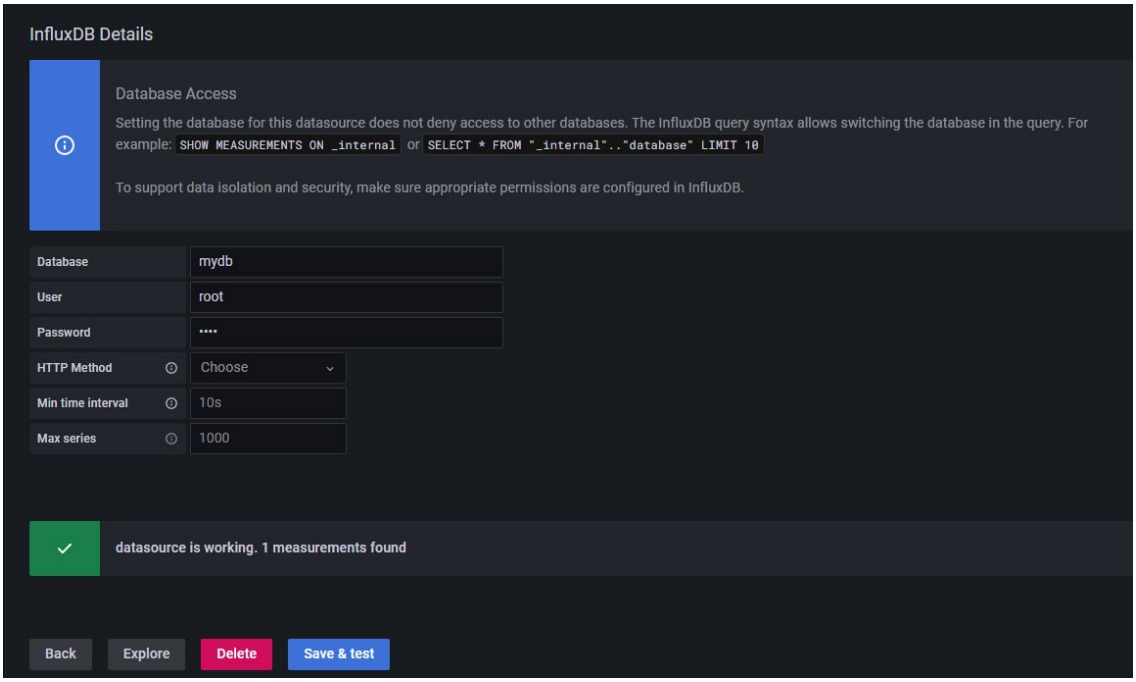
- URL: **http://localhost:8086**

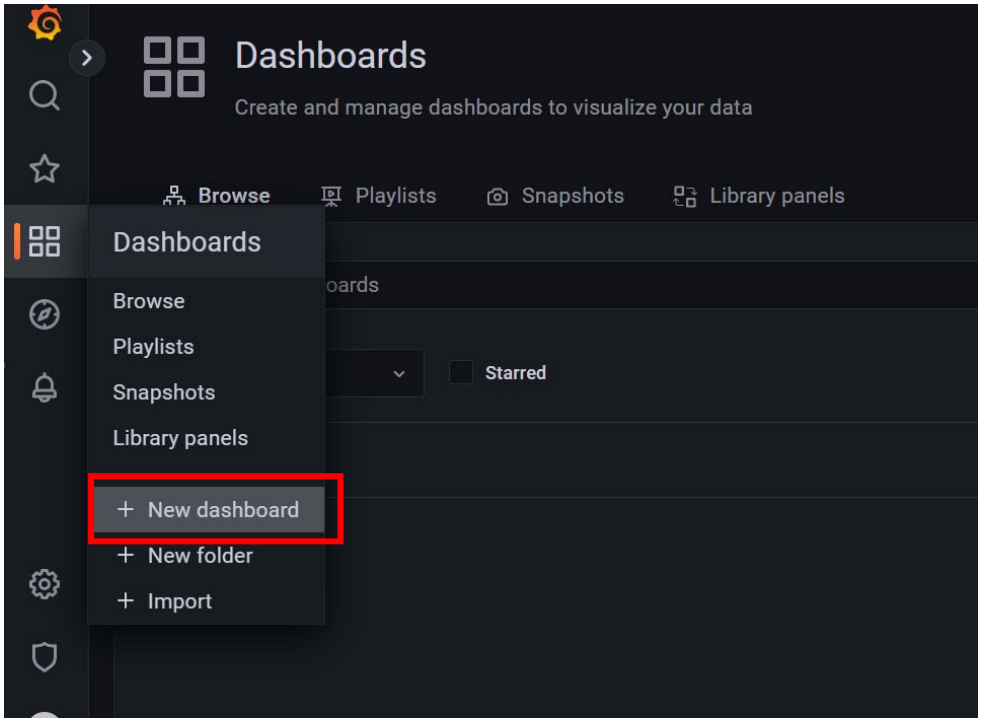


Scroll down to **InfluxDB Details**:

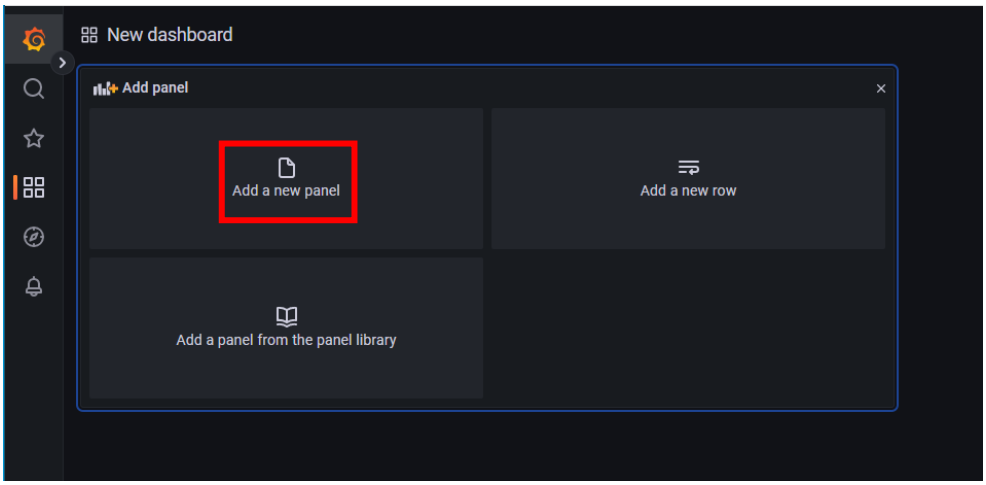
- Database: mydb
- User: root
- Password: root

Click the **Save & Test** button next. If the parameters are set correctly, you will see a message – "**data source is working. 1 meaurement found**".

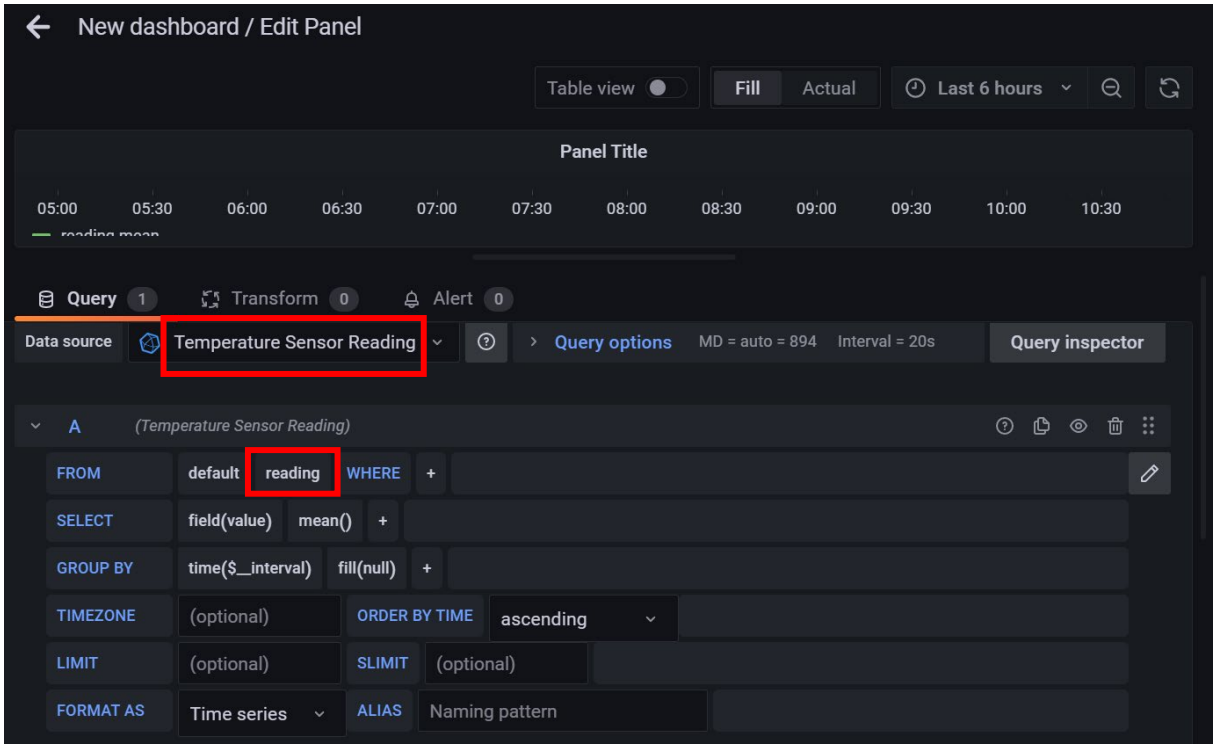




Click the  icon.



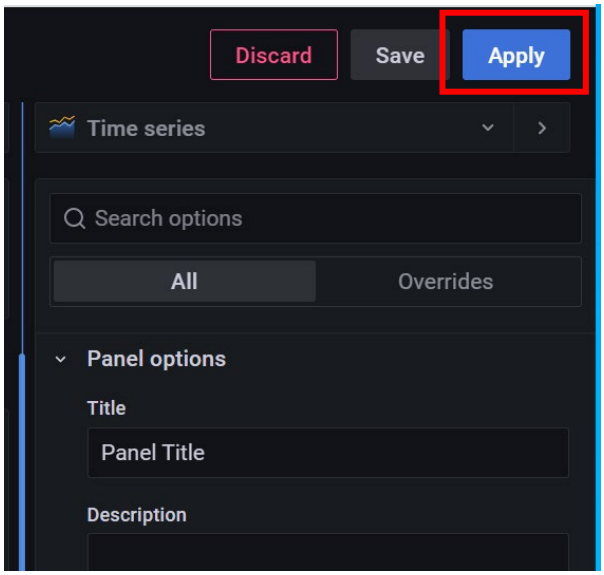
Select **Add a new panel**.



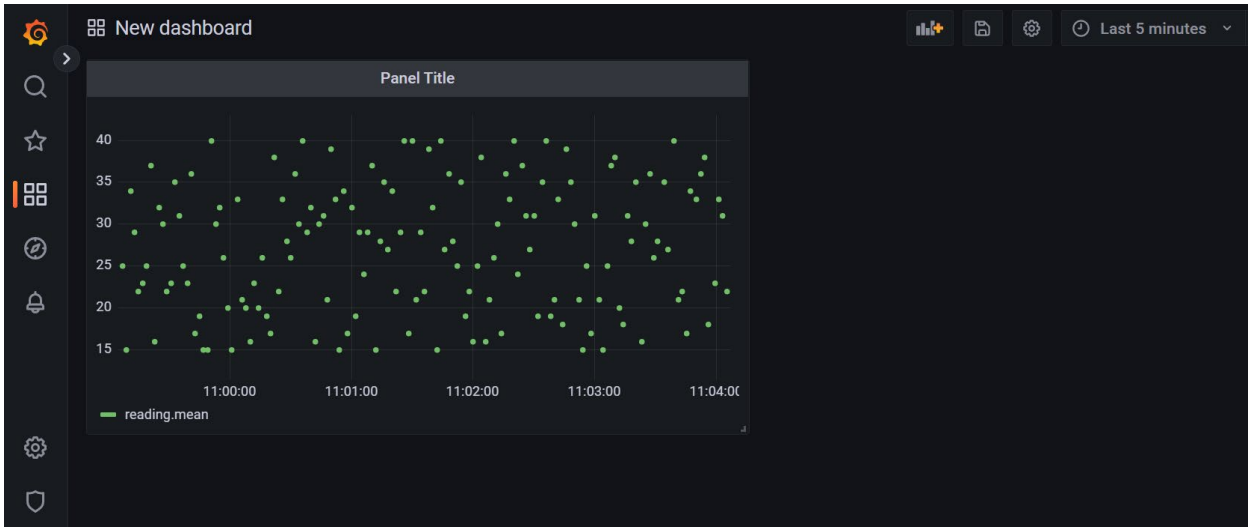
This brings you to **New dasbboard /Edit Panel** page.

Scroll below and Select the **Query** tab page. Provide the following settings:

- **Data Source**: Temperature Sensor Data
- **FROM -> select measurement**: reading



Click **Apply** on the top-right hand corner of the same page when you are done.



The panel should now be populated with the data that you have generated from the running of your Python script **writedb.py** earlier in **Exercise 1**.

Ponder:

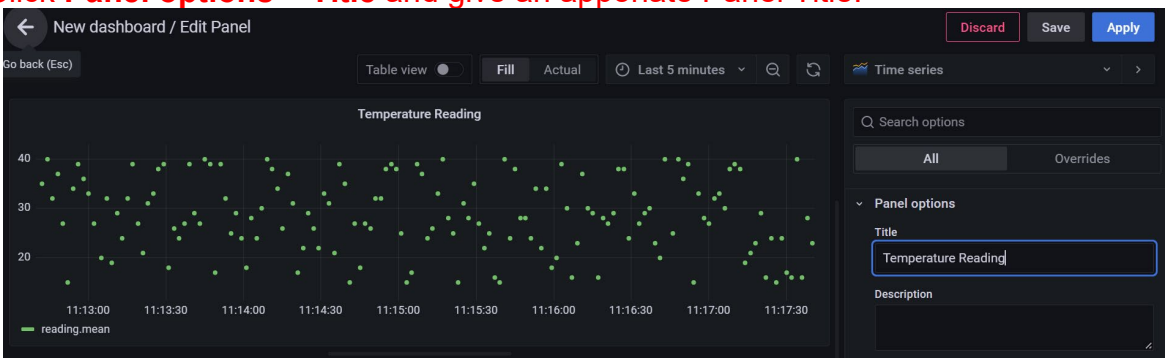
Why is ‘reading’ selected for “select measurement”?

Save your dashboard by clicking on the **save** icon at the top of the screen. Provide a name for the dashboard when prompted.

Select the drop-down menu **Panel Title -> Edit**.

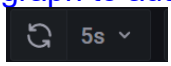
Exercise 2:

1. Give proper labels to the panel title.
   Click **Panel options** > **Title** and give an apporiate Panel Title.



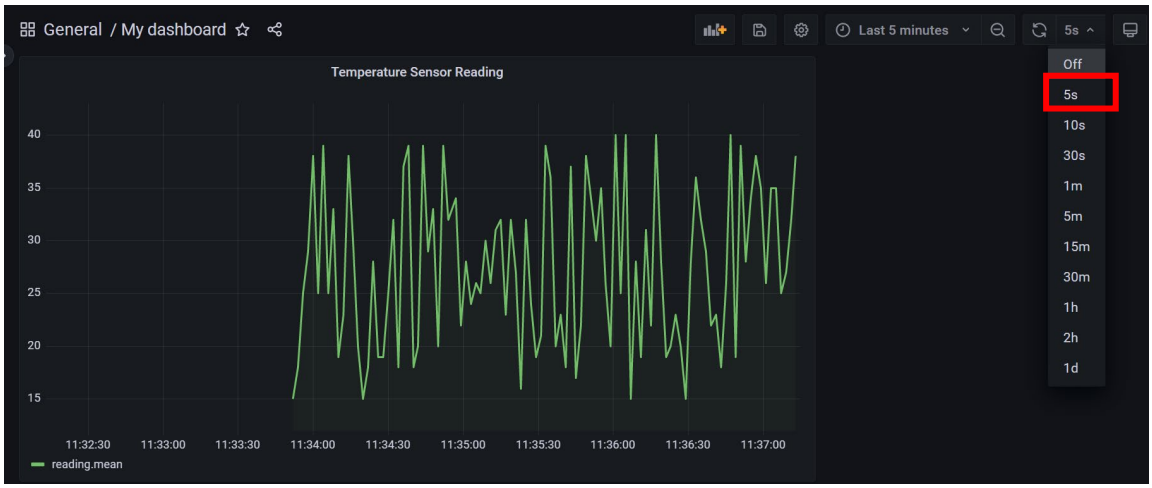
2. Scroll further down to **Connect null values**.
   - Select **Always**

   Observe the graph convert from dotted point to connecting line.
   - Click **Apply**

3. Enable the graph to automatically refresh every 5 seconds.
   - Click the 

icon at top right of screen.

- Select **5s**



4. Allow the Python script to continue running and observe the Grafana chart.
   In terminal window, issue the command: **python3 writedb.py**

3.4 Challenge

Grafana Alerting alerts you to problems in your system moments after they occur. Create, manage, and act on your alerts in a single, consolidated view, and improve the ability to identify and resolve issues quickly.

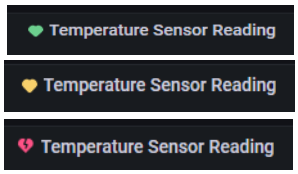
Exercise 3: Setting up an Alert for Monitoring

- Click the **Alert** tab at bottom panel of **My dashboard / Edit Panel**.

Enter the condition for the alert to be activated; such as when the temperature is above certain threshold. What did you observe in the graph when the condition is set?

Do you notice a heart shape symbol besides the panel title which indicate the health of your system after you set the Alert?

- Normal
- Pending
- Alerting



Demonstrate to your tutor.

Online resource to create alert in Grafana:
https://grafana.com/docs/grafana/latest/alerting/
https://www.youtube.com/watch?v=h2daQ8mQ2pM

~ End of Lab 8 ~